Beyond MLR Lab 1: Modeling center as a fixed effect

A multicenter rehabilitation trial

We return to the rehabilitation trial from the lecture. In this simulated study, 120 patients recovering from knee replacement receive either **standard care** or an **enhanced rehabilitation program**. Standard care consists of weekly physiotherapy and home exercises; enhanced care adds a weekly supervised session. The outcome is the distance walked during a six-minute walk test at 12 weeks, measured in meters. A larger distance indicates better walking performance.

The study includes six centers. Within each center, 10 patients are randomly assigned to standard care and 10 to enhanced care. This is a **randomized block design**, with center as the blocking factor. These are simulated teaching data, not observations from an actual clinical trial.

Our question is: **What is the estimated difference in expected walking distance between enhanced and standard care, and how precisely is it estimated?** We will explore differences between centers and fit a model that includes center as a fixed effect.

By the end of this lab, you should be able to visualize treatment and center differences, fit and interpret the fixed-effects model, quantify uncertainty about the treatment effect, and check the model assumptions.

> Working through the lab
>
> Run the code in order in an R script, and record your answers to the questions. The lab page shows the code; you generate the results yourself.

Organize your work

Use one folder per week

Create a folder for this course, with a separate folder for each week. Keep that week's lab scripts and data together so that you can find and rerun your analyses when working on the assignment. For Week 1, use the following structure:

```
BeyondMLR/
  week1/
    lab1.R
    lab2.R
    data/
      rehabilitation_week1.csv
  week2/
```

Create the `week1` folder and its `data` subfolder now. Download the rehabilitation dataset and save it in `week1/data`. Keep the filename unchanged.

Save your code in an R script

An **R script** is a text file with the extension `.R` that records the commands for your analysis. Saving your code lets you repeat the analysis in a new R session and adapt it later.

In RStudio, select **File > New File > R Script** and save the file as `lab1.R` inside `week1`. Copy the R code from this lab into your script as you work through it. Place the cursor on a line, or select several lines, and click **Run** to execute them in the Console. The shortcut is **Ctrl+Enter** on Windows/Linux or **Cmd+Enter** on macOS. Run the code in order, starting with package loading and data import. RStudio's code-editing guide describes these controls.

Add comments beginning with `#` to explain your code and record short answers to the lab questions. Save the script regularly.

Set the working directory

The **working directory** is the folder R uses as the starting point for relative file paths. For example, `"data/rehabilitation_week1.csv"` tells R to look inside the `data` subfolder of its working directory. Relative paths used to save output also start from this folder.

For this lab, set the working directory to `week1`, which contains your script and the `data` folder. Saving or opening a script does not by itself guarantee that R uses that folder as its working directory.

Changing the working directory in RStudio

Select **Session > Set Working Directory > Choose Directory...**, then choose your `week1` folder. Alternatively, with the saved `lab1.R` script active, select **Session > Set Working Directory > To Source File Location**.
Run `getwd()` in the Console to check the selected folder. It should be `week1`, not its `data` subfolder. Check this again when starting a new session. These controls are part of RStudio's working-directory tools.

Setup and data

Install pak once

An R **package** provides additional functions, for example for drawing figures or fitting statistical models. Throughout this course, we will use the package manager **pak** to install the packages we need. It also handles their **dependencies**: other packages they need to work.

First, install `pak` itself by running the following command in the Console. You only need to do this once for your current R installation; skip this step if you already have `pak` installed.

```
install.packages("pak", repos = "https://cloud.r-project.org")
```

Use pak throughout the course

Installing a package saves it on your computer. **Loading** a package makes its functions available in the current R session. Installed packages remain available between sessions, but you need to load them again when you start a new session.

We will follow the same pattern in each lab: load `pak` with `library(pak)`, use `pkg_install()` to install the packages needed for that lab, and then load those packages with `library()`. The pak package manager checks the required packages and their dependencies and keeps already installed versions when they meet the requirements.

For this lab, we need `ggplot2` for figures and `dplyr` for summaries. Add the following code to the beginning of your script and run it. If `pak` asks whether to proceed, confirm the installation and wait for it to finish.

```
library(pak)
pkg_install(c("ggplot2", "dplyr"))
```

Now load the packages for your analysis. Keep these commands in your script so that they run each time you start the lab in a new R session.

```
library(ggplot2)
library(dplyr)
```

Read the data

With `week1` as your working directory and the dataset saved in its `data` subfolder, run the following code. If R cannot find the file, check `getwd()`, the folder containing the CSV file, and its filename.

```
rehab <- read.csv("data/rehabilitation_week1.csv")

# Specify the order of the categories explicitly.
rehab$treatment <- factor(rehab$treatment,
                          levels = c("Standard", "Enhanced"))
rehab$center <- factor(rehab$center,
                       levels = paste("Center", LETTERS[1:6]))

head(rehab)
str(rehab)
table(rehab$center, rehab$treatment)
```

Each row represents one patient. The variables are `center`, `treatment`, and `distance` (six-minute walk distance in meters). A **factor** tells R that a variable represents categories rather than a continuous measurement. The order of the factor levels will matter when we choose how to represent these categories in the model.

> Question 1
>
> Check the allocation table. How many patients are in each treatment group within each center? Why is this allocation useful when comparing the treatments?

Explore treatment and center differences

Create a plot that shows the individual walking distances and the treatment-specific means within each center. Add the overall mean for each treatment as a reference line. This lets us examine center differences while keeping the two treatment groups separate.

First calculate the mean walking distance for each treatment within each center, and then the overall mean for each treatment.

```
center_treatment_means <- rehab |>
  group_by(center, treatment) |>
  summarise(mean_distance = mean(distance), .groups = "drop")

treatment_means <- rehab |>
  group_by(treatment) |>
  summarise(mean_distance = mean(distance), .groups = "drop")
```

Use these summaries together with the individual observations to create the plot.

```
center_plot <- ggplot(rehab,
                      aes(x = center, y = distance, color = treatment)) +
  geom_hline(data = treatment_means,
             aes(yintercept = mean_distance, color = treatment),
             linetype = "dashed", linewidth = 0.7, show.legend = FALSE) +
  geom_point(position = position_jitterdodge(jitter.width = 0.18,
                                             jitter.height = 0,
                                             dodge.width = 0.64,
                                             seed = 20261104),
             alpha = 0.35, size = 1.8) +
  geom_point(data = center_treatment_means,
             aes(y = mean_distance),
             position = position_dodge(width = 0.64),
             shape = 18, size = 4, show.legend = FALSE) +
  scale_color_manual(values = c(Standard = "#173F48",
                                Enhanced = "#D58A3D")) +
  labs(x = "Center", y = "Six-minute walk distance (m)",
       color = "Treatment") +
  guides(color = guide_legend(override.aes = list(alpha = 1, size = 3))) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "top")

center_plot
```

The small points are individual observations. They are shifted slightly along the horizontal axis to make overlapping points easier to see; their walking distances are unchanged. The diamonds show the observed treatment means within each center. The dashed lines show the overall treatment means across all six centers.

Question 2

Does the plot suggest substantial differences in walking distance between centers? Support your answer using the center-specific means and the variation among patients within centers.

Specify the model and code the factors

Specify the model

Let Y_{ij} be the outcome for patient i in center j, and let T_{ij} indicate the patient's treatment. We include an intercept β_0, a treatment coefficient β_1, and a fixed contribution c_j for each center:

$$Y_{ij} = \beta_0 + \beta_1 T_{ij} + c_j + \varepsilon_{ij}, \qquad \varepsilon_{ij} \overset{\text{iid}}{\sim} N(0, \sigma^2).$$

Without a constraint, increasing the intercept and decreasing every center contribution by the same amount would leave the model's predictions unchanged. To define these parameters uniquely, we require the six center contributions to sum to zero:

$$c_A + c_B + c_C + c_D + c_E + c_F = 0.$$

The model assumes that the expected treatment difference is the same in every center: it contains no treatment-by-center interaction. After accounting for treatment and center, the remaining errors are assumed to be independent and identically distributed Normal variables with mean zero and common variance.

To fit this model in R, we explicitly set how each factor is represented numerically. We use **dummy coding** for treatment to define T_{ij} and **effects coding** for center so that R estimates the center deviations specified above.

Code treatment using dummy coding

With **dummy coding**, one category is the reference. We use standard care as the reference and represent treatment by a single indicator:

Treatment	Indicator T_{ij}
Standard	0
Enhanced	1

Within center j, the expected standard-care outcome is $\beta_0 + c_j$, and the expected enhanced-care outcome is $\beta_0 + \beta_1 + c_j$. The treatment coefficient β_1 is therefore the enhanced-minus-standard difference.

```
# The first factor level, Standard, is the reference.
contrasts(rehab$treatment) <- contr.treatment(levels(rehab$treatment))
contrasts(rehab$treatment)
```

`contr.treatment()` creates the dummy-coding matrix. Assigning it with `contrasts()` tells R to use this coding when fitting a model with the treatment factor. Printing the matrix shows how each treatment is represented.

Code center using effects coding

The sum-to-zero constraint makes each c_j a deviation from the equally weighted average across centers, comparing patients receiving the same treatment. With standard care coded 0, the intercept β_0 is **the expected standard-care outcome averaged equally across the six centers**. It does not average over the two treatments. In this balanced trial, equal weighting of centers also agrees with weighting by their patient counts.

We use **effects coding** to estimate these deviations directly. With dummy coding for center instead, the intercept would describe standard care in a reference center, and the other center coefficients would describe differences from that center.

```
contrasts(rehab$center) <- contr.sum(levels(rehab$center))
contrasts(rehab$center)
```

`contr.sum()` creates the effects-coding matrix. Inspect the result: there are six rows, one per center, but only five columns. Centers A–E each have a 1 in their own column and 0 in the others. Center F has -1 in all five columns. Its contribution is therefore minus the sum of the other five contributions; it is not set to zero. Only five center coefficients are needed because the sum-to-zero constraint determines the sixth.

Changing the center coding changes the interpretation of the intercept and center coefficients, but does not change the fitted outcomes or the estimated treatment difference and its standard error. Setting the coding explicitly ensures that the analysis does not depend on previously changed global R options.

Fit and interpret the model

Fit the model and read its output

```
model_fixed <- lm(distance ~ treatment + center, data = rehab)
summary(model_fixed)
```

In an R model formula, the outcome is on the left of `~`, and predictors are on the right. An intercept is included automatically. R uses the coding we assigned to the treatment and center factors.

In the output, `(Intercept)` estimates β_0, and `treatmentEnhanced` estimates β_1. The five center coefficients are named `center1` to `center5`, corresponding to Centers A–E. For example, a positive `center1` indicates a higher expected outcome in Center A than the center average, for either treatment. The model-predicted standard-care outcome in Center A is the intercept plus `center1`.

> Question 3
>
> Interpret the intercept, the treatment coefficient, and `center1`. How would you recover the contribution for Center F? Why would the intercept have a different interpretation if we used dummy coding for center?

Treatment effect and uncertainty

In `summary(model_fixed)`, the row `treatmentEnhanced` gives the estimated enhanced-minus-standard difference, its standard error, and a test against a zero difference. The residual standard error at the bottom of the output estimates the individual-error standard deviation after accounting for treatment and center.

Use `confint()` to obtain the 95% confidence interval for the treatment coefficient, and `sigma()` to extract the residual standard deviation.

```
confint(model_fixed, "treatmentEnhanced")
sigma(model_fixed)
```

The standard error of the treatment coefficient describes uncertainty about the estimated treatment difference. The residual standard deviation describes the remaining variation in individual walking distances. Both are expressed in meters.

Question 4

What are the estimated enhanced-minus-standard difference, its standard error, and its 95% confidence interval? What is the residual standard deviation, and how does its interpretation differ from that of the treatment coefficient's standard error?

Check the model assumptions

As in ordinary linear regression, residual plots help assess whether the error distribution and variance assumptions are plausible.

```
plot(model_fixed, which = 1)  # Residuals versus fitted values
plot(model_fixed, which = 2)  # Normal Q-Q plot
```

The first plot can reveal changes in residual spread or patterns left in the mean. The second compares the residual distribution with a Normal distribution. Residuals are calculated from the fitted model; they are estimates of the unobserved errors.

Question 5

Do these plots suggest clear violations of the Normality or constant-variance assumptions? Describe the patterns that support your answer.